EXOPLANET HABITABILITY PREDICTOR

Milestone 3: Theoretical Documentation for Backend and Frontend Development

A clean, responsive, and production-oriented deployment layer for ML-based habitability prediction

Milestone 3 Document

Module	Core Focus	Format	Prepared For
Module 5 & 6	Flask backend + responsive frontend UI	PDF report	Academic submission

Theoretical coverage includes API design, model deployment, client-server communication, user interface design, and error handling.

Design principle: the backend must remain reliable and predictable, while the frontend must stay simple, responsive, and easy to use.

1. Objective and System Overview

The goal of Milestone 3 is to convert the trained machine learning pipeline into an accessible application with two connected layers: a Flask-based backend and a browser-based frontend. Together, these layers transform the model from a notebook artifact into an interactive prediction system.

- The backend acts as the service layer that loads the trained model, processes requests, and returns predictions in JSON format.
- The frontend acts as the presentation layer that captures user input, sends it to the backend, and displays the output in a readable form.
- The overall system follows a client-server model, where the browser remains lightweight and the prediction logic remains centralized on the server.

System Layer Summary

Layer	Main Role	Key Benefit
Backend	Runs the model and exposes REST endpoints	Keeps logic secure and reusable
Frontend	Collects input and shows results	Improves usability and accessibility
ML Model	Generates habitability predictions	Delivers scientific decision support

2. Backend Theory

The backend is responsible for serving the machine learning model through a REST API. It hides the internal implementation of the model and exposes a small set of predictable endpoints that can be called by a web page, a testing tool, or any other client.

- Flask is used because it is lightweight, easy to configure, and well suited for small to medium-sized APIs.
- The API accepts JSON requests so that structured numerical and categorical values can be passed without manual parsing.
- The model is loaded once when the application starts, which reduces repeated disk access and improves runtime efficiency.
- Utility functions are typically separated into a helper file so that data preparation and validation remain cleanly organized.

Backend Components Table

Backend Component	Purpose	Practical Importance
app.py	Starts the Flask server and defines routes	Controls the API behavior
utils.py	Validates input and prepares features	Prevents malformed requests
final_model.pkl	Stores the trained estimator/pipeline	Enables deployment without retraining
/predict	Returns habitability prediction and score	Supports interactive inference
/rank	Returns ranked exoplanet list	Useful for screening candidates

2.1 Request Handling and Validation

- Every incoming request should be checked for completeness before the model is executed.
- Missing or invalid fields should produce a human-readable error message instead of a crash.
- Validation protects the backend from type mismatches, empty payloads, and unexpected client behavior.
- This step is especially important for scientific applications because input quality directly affects prediction quality.

2.2 Prediction Endpoint

- The /predict endpoint is the main inference route and is generally implemented using the POST method.
- The request body contains the exoplanet parameters as JSON so the server can reconstruct the feature vector.
- The response typically includes the predicted class, the habitability probability, and a short status message.
- Using JSON keeps the endpoint language-agnostic and easy to connect to any frontend or testing tool.

2.3 Ranking Endpoint

- The /rank endpoint is used to return exoplanets sorted by predicted habitability score.
- This is useful when the goal is not only classification but also prioritization of the most promising planets.
- A ranked output allows the user to focus on the strongest candidates first, which is more practical than viewing raw probabilities alone.

2.4 Error Handling and Cross-Origin Access

- The backend must return meaningful errors for invalid requests, missing data, and model-loading failures.
- Cross-Origin Resource Sharing (CORS) is often enabled so that a browser-based frontend can communicate with the backend without blocking issues.
- A controlled error response improves robustness and helps the frontend display user-friendly messages.

Backend design rule: keep the API stateless, deterministic, and easy to test. Each request should be independent and should return the same output for the same input.

2.5 Backend Testing Strategy

- The home route is used as a quick health check to confirm that the server is running.
- The prediction route can be tested using browser tools, Postman, curl, or the frontend UI.
- Testing verifies both functional correctness and whether the model loads correctly in the deployment environment.
- A successful test should confirm that the server responds quickly and does not crash on valid requests.

Backend Processing Flow

Stage	Backend Action	Expected Output
1	Receive JSON request	Structured payload
2	Validate required fields	Clean input or error message
3	Prepare feature frame	Model-ready input
4	Run prediction	Binary habitability class
5	Return score	Probability or confidence value

3. Frontend Theory

The frontend is the user-facing interface of the project. It allows a person to enter exoplanet data without writing code and turns the server response into a clear visual output. The goal is to keep the experience simple, responsive, and professional.

- HTML provides the structure for the form, labels, output cards, and feedback messages.
- Bootstrap is used to speed up layout design and ensure that the page adapts well to different screen sizes.
- CSS is used to refine spacing, colors, shadows, and typography so the interface looks polished.
- JavaScript handles asynchronous communication with the backend and updates the page without reloading it.

Frontend Components Table

Frontend Component	Purpose	User Benefit
index.html	Defines the input form and result area	Clear structure
style.css	Applies custom visual styling	Readable and consistent appearance
script.js	Sends data and processes responses	Interactive experience
Bootstrap	Provides responsive layout utilities	Works across devices

3.1 Form Design and Input Quality

- The input form should mirror the backend's expected feature names and data types.
- Placeholders and labels help the user understand what kind of value is required for each field.
- Client-side validation can prevent obviously invalid values from being sent to the API.
- A well-designed form reduces the chance of server-side errors and makes the application easier to use.

3.2 Asynchronous Communication

- The frontend should use Fetch API or Axios to send data to the backend asynchronously.
- Asynchronous requests keep the page responsive because the browser does not need to reload after every submission.
- The backend response is parsed as JSON and then inserted into the result cards or alerts.

3.3 Result Display and User Feedback

- Prediction output should be shown with clear labels such as habitability status and habitability score.
- Success and error alerts make the interface easier to interpret for non-technical users.
- Cards and tables provide a clean and visually organized way to present the final result.

3.4 Styling and Responsiveness

- The page should remain readable on desktop, tablet, and mobile screens.
- Consistent padding, rounded cards, and subtle shadows improve visual hierarchy without overwhelming the user.
- A restrained color palette keeps the application professional and aligned with academic or technical use.

Frontend design rule: make the interaction feel immediate. A user should be able to submit data, see feedback quickly, and understand the result without confusion.

4. Backend and Frontend Integration

The backend and frontend are connected through the HTTP request-response cycle. The frontend is responsible for collecting and formatting the input, while the backend is responsible for validating the data and producing the machine learning output.

Integration Workflow Table

Step	Frontend Role	Backend Role	Outcome
1	User fills the form	—	Input data is prepared
2	JavaScript serializes JSON	—	Request is ready to send
3	Browser sends POST request	Flask receives the payload	Server-side processing begins
4	—	Model predicts habitability	Prediction and score are generated
5	Frontend renders response	Backend returns JSON	User sees the final result

4.1 Integration Principles

- The frontend and backend must agree on the same field names to avoid mismatched requests.
- The API should return stable JSON keys so the frontend can read and display the values consistently.
- Separating display logic from prediction logic keeps the application easier to maintain and debug.
- Any backend failure should be caught and shown to the user as a meaningful message rather than a browser crash.

5. Conclusion

- The backend provides a reliable machine learning service layer using Flask and JSON-based endpoints.
- The frontend provides a clean and responsive interface for user input and prediction display.
- Together, they turn the trained exoplanet habitability model into a complete web application.
- The final result is a deployable system that is suitable for demonstrations, testing, and further extension.

6. Quick Summary of Design Strengths

Aspect	Strength
Usability	Simple form-driven interface with immediate feedback
Maintainability	Clear separation between backend logic and frontend presentation
Reliability	Validation and error handling reduce crashes
Presentation	Professional typography, symmetric spacing, and consistent color accents

End of Milestone 3